Forge Conductor - Dossier

Derived reading copy. Markdown remains the source of record. No source file is modified by this build.

Ridgeboundia / DeerFlow Forge Agent Program

Status: research + execution evidence + implementation plan. **No production repair has been applied.** Date: 2026-11-05. Owner: Riley.

This is the master entry point for the agent-harness work. It explains what exists, what is broken, what we are building, the six integrations, eight Python changes, eight agent/tool/skill changes, the goal evaluation, and how to read the evidence without drowning in it.

> **Read next:** [`GOALS.md`](GOALS.md) -> [`ARCHITECTURE.md`](ARCHITECTURE.md) ->
> [`execution-wave/09-GOAL-GAP-PLAN.md`](execution-wave/09-GOAL-GAP-PLAN.md) -> [`PLAIN-ENGLISH.md`](PLAIN-ENGLISH.md).
> Diagrams: [`diagrams/index.html`](diagrams/index.html).

---

1. The outcome we are building

A hosted **Forge Conductor** on the existing DeerFlow 2.2 + DeepSeek V4.1 Flash runtime that can:

1. take a print/vector brief;
2. choose exact Craft command ids from a pinned catalogue;
3. run on copies through a typed CLI/MCP gateway;
4. verify artifact hashes, dimensions/DPI, domain gates, an independent render and a blind visual critic;
5. record a hash-chained episode with evidence;
6. propose a lesson/skill only after repeated verified success and human approval;
7. never publish to the storefront without explicit human approval.

The first release is a **production operator under human art direction**, not an autonomous artist, not a new generic superagent, and not a claim of "better than human."

2. What we know now (the short truth)

Area	Finding
VPS	2 vCPU / 7.9 GB / 36 GB free, no GPU/display; DeerFlow 2.2 already running
Model	`deepseek-flash` ("DeepSeek V4.1 Flash"), 1,048,576 ctx, 8,192 output; key validity unverified
Craft apps	VectorCraft 0.21.1 + PhotoCraft 0.20.2 installed; Film/Design/Effect **absent**
Execution	`/opt/pod-pipeline/bin/*-cli` are **stubs exiting 127**; `routes.py` argv is invented
MCP	Craft MCP works standalone (33/26 tools) but is **not registered** in DeerFlow
Persistence	`pod` DB empty/schema-only; **no** episode/action/eval/skill tables
Learning	DeerMem facts + skill-evolution host exist; **no** full learner or quality evaluation
Publish	`fourthwall_template_id` unset for all five products; no approval queue
Evidence	README's live-E2E claim is historical/unreproducible in the current state
Assets	39 skills, 400 answers, 6 examples, 8 validated diagrams, 131-file package

The key sentence: **the model/harness is present; the Craft execution path and learning evidence are not.** Phase 0 starts there.

3. Six integrations

#	Integration	What it connects	Why it is needed	Acceptance gate
I1	**Craft launcher + argv contract**	`/opt/craft/*/bin` -> `deploy/craft-bin` -> `routes.py`	The pipeline cannot execute a Craft command today	one real VectorCraft trace + one real PhotoCraft retouch
I2	**Craft MCP bridge**	VectorCraft/PhotoCraft stdio MCP -> DeerFlow `mcpServers`	Gives the agent native 33/26-tool surfaces without shell guessing	`tools/list` + `command_batch`; dependent PhotoCraft calls awaited
I3	**Forge Conductor service**	Console/DeerFlow -> `forge-conductor:8092` -> queue/ledger	Owns planning, budgets, episode state and recovery	replay one episode from ledger; no source mutation
I4	**Evidence verifier + blind critic**	artifacts -> gates -> render -> tile metrics -> critic	Exit 0/`ok:true`/screenshots are not proof	5-check composite + human rubric; missing evidence = UNVERIFIED
I5	**Human approval + publish**	Forge approval -> Fourthwall product state	Publishing is irreversible and currently misconfigured	approval bound to artifact hash; template ids set; no auto-publish
I6	**Learning + evaluation loop**	episodes -> retrieval -> candidate skill -> regression -> active	"Learns" must mean a measured, reversible change	calibrated promotion + A/B; null result disables memory

I1 - Launcher + argv contract (P0)

- **Files:** `blueprint/files/deploy/craft-bin/*-cli`, `blueprint/files/pod_pipeline/routes.py`, `deploy/setup-pipeline.sh`.
- **Measured contract (do not revert to the old ASSUMPTION):**
- VectorCraft: `run --in F --cmd select.all --cmd imageTrace.make --params '{"params":{...}}' --cmd imageTrace.expand --export out.svg`; export does **not** create parent dirs.
- PhotoCraft: `run FILE|--new --cmd ID --params J --out F`; creates parent dirs.
- **Refusal:** absent app -> `E_TOOL_NOT_INSTALLED`, not a fake trace.
- **Rollback:** restore the original launchers and `routes.py` from backup.

I2 - Craft MCP bridge (P0)

- **Files:** DeerFlow `extensions_config.json`; optional stdio bridge; read/write roots.
- **Rules:** VectorCraft `mcp --headless --automation-read/write-root`; PhotoCraft `mcp` without `--headless` + roots; await each `tools/call` (PhotoCraft can interleave).
- **Gate:** `tools/list` returns 33/26; a real batch produces an artifact.

I3 - Forge Conductor service (P1)

- **Files:** new `blueprint/files/forge/*`, `sql/schema_forge.sql`, `deploy/systemd/forge-conductor.service`, `deploy/systemd/forge-worker.service`.
- **DB:** 13 tables (episode, event, handoff, tool_call, artifact, gate, critique, eval, job, skill, budget, approval, transition).
- **Gate:** 18 SQL assertions, 33 Python tests, SKIP LOCKED two-worker test; episode replay.

I4 - Evidence verifier + blind critic (P1)

- **Files:** `forge/verifier.py`, `forge/critic.py`, gate adapters.
- **Ladder:** process -> artifact/hash -> domain gates -> independent render -> tile/crop metrics -> blind critic -> human.
- **Measured:** global SSIM catches 7/15; tile-min 13/15 but 3/4 false alarms; 5-check composite **15/15 recall, 0/4 false alarms** on the synthetic corpus.
- **Gate:** missing metric -> `UNVERIFIED`; critic cannot override a failed gate.

I5 - Human approval + publish (P1)

- **Files:** `forge/approval`/`forge_publish.py`, `config/products.json`, Fourthwall client.
- **Rules:** approval table binds to artifact hash + scope; template/area validated; no autonomous publish; product state read back.
- **Gate:** approval reused -> blocked; missing template -> named refusal; no store mutation in tests.

I6 - Learning + evaluation (P2/P3)

- **Files:** `forge/memory.py`, `forge/promotion.py`, `forge/eval.py`, DeerFlow memory/skill config.
- **Rule:** L1/L2 only (retrieval/skills), no fine-tune/RL. Promotion requires a declared horizon, Wilson lower bound above baseline, golden regression and human approval. **3/3 is not enough** (Wilson LB 0.4385; the old rule was rejected by simulation).
- **Gate:** A/B on held-out goldens; null result disables memory; rollback pointer.

4. Eight small Python enhancements

These are **planned**, not applied. Each is written to be independently testable.

#	File / function	Change	Test
P1	`pod_pipeline/routes.py` `trace_step` / `raster_fallback_step`	replace invented `trace`/`--input` argv with measured `run` contracts	`test_routes.py` + one live trace/retouch
P2	`pod_pipeline/orchestrator.py` `_run_step` / preflight	check binary exists, app version, catalogue hash; `E_TOOL_NOT_INSTALLED`	missing-binary error path
P3	`tools/craft_trace.py` gate assembly	make `retyped_text_present` non-vacuous; record artifact path/hash; gate status before/after save	0-box text fixture must fail; text fixture must pass
P4	`tools/prep_print_art.py` report	stop treating `ok:true` as a pass; expose warning-only upscale and coverage; add `verdict`	3.6x upscale test must be `refused_for_print`
P5	`pod_pipeline/runner.py` / orchestrator	hash inputs before/after; flag source mutation; atomic artifact copy	mutate a fixture; episode must fail
P6	`pod_pipeline/db.py` / new `forge/repo.py`	idempotency key, duplicate replay, unique constraints, queue lease	duplicate call = one effect; crash/requeue
P7	`pod_pipeline/worker.py`	lease expiry/reaper/recovery with bounded scans	kill worker mid-run; reclaim once
P8	`pod_pipeline/orchestrator.py` / new `forge/cost.py`	persist token/cost per step/episode; enforce caps	cap exceeded returns named refusal, no silent truncation

5. Eight agent/tool/skill enhancements

These are changes to how agents use the stack; most are documentation/protocol now, code/tooling later.

| # | Skill/tool | Enhancement | Why |

|---|---|---|---|

| **A1** | `craft-agent-operator` | mandatory evidence contract + no screenshot/exit-code proof + episode record | prevents "it looked good" shipping |

| **A2** | `agent-learning-loop` | calibrated promotion/demotion/regression, corrected 3/3 rule | simulation showed the original rule cannot promote |

| **A3** | `visual-verification-critic` | 5-check composite, black+white render, region/tile evidence | global SSIM misses 4% defects |

| **A4** | `forge-harness-architect` | Phase 0 app/binary preflight; VC/PC-only scope | prevents designing for absent apps |

| **A5** | `harness-code-adapter` | donor provenance/pin, adapter contract, no verbatim code | code stitching without a fork |

| **A6** | `research-corpus-navigator` | map 400 answers + claim/errata; one-slice loading | stops context flooding |

| **A7** | `craft-mcp-drive` | await dependent calls; auth/loopback; stdio vs HTTP gap | PhotoCraft interleaving + exposed MCP |

| **A8** | `apparel-print-master` / publish skills | bind approval to artifact hash; template/rights preflight | no irreversible publish on a stale master |

6. Phase plan and gates

| Phase | Deliverable | Gate |

|---|---|---|

| **0** | launchers, argv, MCP registration, 10 goldens, README/AGENTS correction | one real trace + retouch; no stub; expected artifacts |

| **1** | Forge skeleton, verifier, approval, queue | replay from ledger; duplicate/refusal/crash tests |

| **2** | blind critic, tile metrics, human rubric | 15/15 synthetic recall / 0/4 false alarms; missing -> UNVERIFIED |

| **3** | retrieval + A/B | measured gain or null; memory disabled on null |

| **4** | skill promotion/rollback | calibrated horizon + regression + human; rollback drill |

| **5** | print + second app + 50 goldens | no unsafe publish; measured cost |

| **6** | security/backup/hardening | restore drill; key metadata check; MCP auth |

Estimate: 6-10 weeks for Phase 0-2; this is an `ESTIMATE`, not a measured schedule.

7. Goal evaluation (summary)

| Goal | Status | Evidence | What moves it |

|---|---|---|---|

| G1 execute Craft | **BLOCKED** | stubs + invented argv | I1/P1/P2 |

| G2 native tools | **PARTIAL** | MCP works standalone, not registered | I2/P6 |

| G3 verify quality | **PARTIAL** | gates exist, can lie; no critic integration | I4/P3/P4 |

| G4 learn | **NOT STARTED (L0)** | no ledger/eval | I3/I6/P6-P8 |

| G5 host on VPS | **FEASIBLE** | 2 vCPU, model running | I3 + caps |

| G6 safe publish | **BLOCKED** | no approval, templates unset | I5/A8 |

| G7 truth | **IMPROVING** | claim register; README correction pending | docs/ERRATA |

Full detail: [`GOAL-EVALUATION.md`](GOAL-EVALUATION.md).

8. What is possible / what is best

Possible now: repair the execution path, run one real trace/retouch, record evidence, verify dimensions/DPI/gates, build the ledger/queue, and prove replay/idempotency. **Best first move:** I1 + I2 + I3, not GUI or training.

Not possible now: all Craft apps, autonomous publish, fine-tuning, RL, "better than human," or a production claim that the current agent learns.

Best architecture: DeerFlow 2.2 as front door/runtime + Forge Conductor as third plane + native Craft CLI/MCP + Postgres ledger/queue + DeerMem facts + versioned skills. Do not build a new generic harness or vendor a whole donor repo.

9. Evidence and files

```
| Item | Where |
|---|---|
| Goals/non-goals | [`GOALS.md`](GOALS.md) |
| Architecture | [`ARCHITECTURE.md`](ARCHITECTURE.md) |
| Integration map | [`INTEGRATIONS.mmd`](INTEGRATIONS.mmd) + SVG/PNG |
| Eight system diagrams | [`diagrams/`](diagrams/index.html) |
| Integration map (ninth) | [`INTEGRATIONS.mmd`](INTEGRATIONS.mmd) + SVG/PNG |
| PDF reading copies | [`PDF-CATALOGUE.md`](PDF-CATALOGUE.md) - 29-page binder in `docs/pdf/` |
| Goal->code gap + decision plan |
[`execution-wave/09-GOAL-GAP-PLAN.md`](execution-wave/09-GOAL-GAP-PLAN.md) |
| Recon ledger | [`execution-wave/RECON-LEDGER.md`](execution-wave/RECON-LEDGER.md) |
| Claim register | [`execution-wave/10-CLAIM-REGISTER.md`](execution-wave/10-CLAIM-REGISTER.md) |
| Independent verification |
[`execution-wave/08-VERIFICATION-INDEPENDENT.md`](execution-wave/08-VERIFICATION-INDEPENDENT.md) |
| First 200 | [`REPORT.md`](REPORT.md) |
| Second 200 | [`wave2/08-200-QUESTIONS.md`](wave2/08-200-QUESTIONS.md) |
| Six examples | [`examples/`](examples/README.md) |
| PDF catalogue | [`PDF-CATALOGUE.md`](PDF-CATALOGUE.md) |
| Plain English | [`PLAIN-ENGLISH.md`](PLAIN-ENGLISH.md) |
```

10. Truth rules

- A design is not an implementation; `PROPOSED` is not `VERIFIED`.
- Exit 0, `ok:true`, a screenshot or a README claim is not proof.
- The 400 answers are a fixed corpus; re-fetch before changing policy.
- No production file/DB/service has been changed by this research wave.
- "Better than human" is unsupported; no blind study exists.
- FilmCraft/DesignCraft/EffectCraft are absent and out of the current scope.
- The key is configured by name only; validity is unverified until the zero-spend metadata check.

11. Next action

Approve **Phase 0**. Apply I1/I2 on a staging copy, run one real trace and one real retouch, write the evidence to the ledger, then decide whether Phase 1 is worth building. Do not start with fine-tuning, GUI automation or autonomous publishing.

Architecture

Status: target architecture + as-is architecture. Design, not deployed. Date: 2026-11-05.

This document answers: where the agent lives, what it calls, what it owns, what it must never own, how evidence flows, and how a run is recovered. For the short version see [`README.md`](README.md).

---

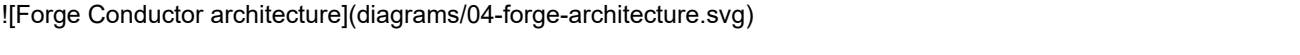
1. As-is (what is actually running)

```
Operator/SSH
?
??? console (127.0.0.1:8091) ?? pure viewer, no agent loop
??? pod-mcp (0.0.0.0:8765) ?? 3 tools, unauthenticated
? ??? pod worker/orchestrator ?? deterministic Python pipeline
? ??? /opt/pod-pipeline/bin/*-cli ?? STUBS, exit 127
?
??? DeerFlow 2.2 gateway/UI ?? model deepseek-flash, subagents, DeerMem, checkpoints
??? only registered MCP: forge (3 pipeline tools)
```

Real Craft binaries exist at `/opt/craft/vectorcraft/.../vectorcraft-cli` and `/opt/craft/photocraft/.../photocraft-cli`; their MCP servers work standalone but are not registered.
No episode/eval/skill tables exist in `pod`; DB is empty.

The pipeline's argv is invented (`routes.py:73-96`), so the chain breaks at the first Craft step.
The README's live-E2E sentence is historical, not current.

2. Target (Forge Conductor)




```
DeerFlow 2.2 (front door, sessions, subagents, DeerMem, checkpoints)
? TaskEnvelope (brief, rights, app/version, output contract, gates, budget)
????????????????????????????????????????????????????????????????????????
? Forge Conductor - 127.0.0.1:8092 ?
? supervisor - budget - arbitration - plan DAG ?
? ?? art director ?? vector operator ?? raster operator ?
? ?? print verifier ?? blind visual critic ?? memory curator ?
? ?? typed tool gateway (allowlist, roots, idempotency) ?
? ?? verifier (artifact/hash/gates/render) ?
? ?? episode ledger + eval/promotion ?
????????????????????????????????????????????????????????????????????????
? ? ?
/opt/craft CLI/MCP Postgres `forge` artifact store
VectorCraft/PhotoCraft episode/event/... SHA-256 + gate reports
?
/opt/pod-pipeline (deterministic, unchanged) - Fourthwall (approval only)
```

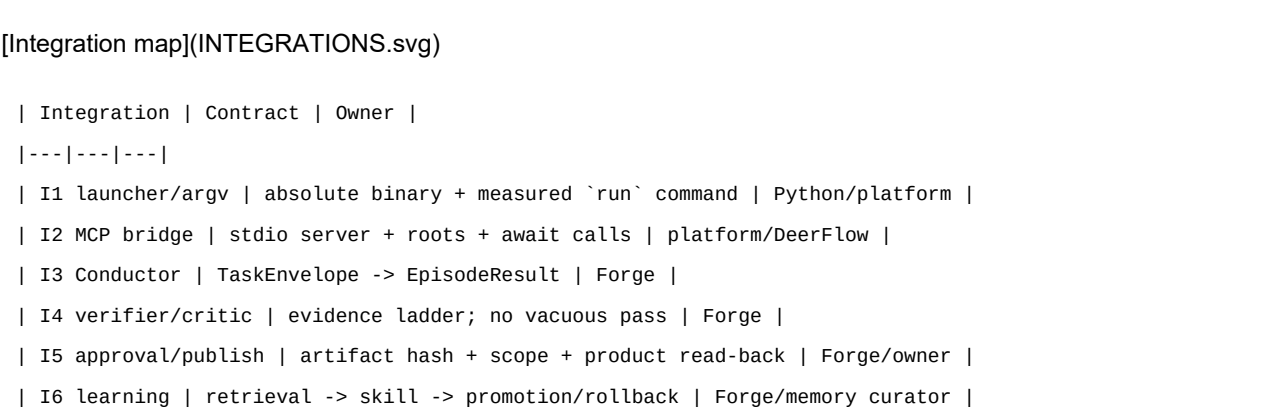
Plane boundaries

```
| Plane | Owns | Never owns |
|---|---|---|
| Console | read-only observation | agent loop, DB writes |
| `pod_pipeline` | deterministic asset run/publish steps | LLM planning, creative choice |
| DeerFlow | conversation, sessions, DeerMem facts, subagents, checkpoints | Craft command semantics, final quality verdict |
| **Forge** | task envelope, planning, tool calls, evidence, ledger, promotion | model weights, storefront authority |
| Craft apps | document truth, command semantics | agent memory |
```

3. Request lifecycle

1. **Brief** - user/operator/DeerFlow submits a `TaskEnvelope`.
2. **Preflight** - rights, app/version/catalogue hash, output contract, budget, approval needed.
3. **Retrieve** - bounded prior episodes/skills with the same app/version/task family.
4. **Plan** - command ids from the pinned catalogue; a gate per step; stop condition.
5. **Execute** - typed gateway; one step at a time on a copy; idempotency key; await every MCP reply.
6. **Verify** - process -> artifact/hash -> gates -> independent render -> tile/crop -> blind critic -> human rubric. Missing evidence = `UNVERIFIED`.
7. **Record** - append-only hash-chained episode and artifact index.
8. **Reflect** - only verified episodes; candidate lesson/skill with evidence ids.
9. **Promote** - calibrated horizon + Wilson LB > baseline + golden regression + human approval; pointer flip; rollback retained.
10. **Publish** - only after artifact-hash-bound approval; verify storefront read-back.

4. Integration map



```
| Integration | Contract | Owner |
|---|---|---|
| I1 launcher/argv | absolute binary + measured `run` command | Python/platform |
| I2 MCP bridge | stdio server + roots + await calls | platform/DeerFlow |
| I3 Conductor | TaskEnvelope -> EpisodeResult | Forge |
| I4 verifier/critic | evidence ladder; no vacuous pass | Forge |
| I5 approval/publish | artifact hash + scope + product read-back | Forge/owner |
| I6 learning | retrieval -> skill -> promotion/rollback | Forge/memory curator |
```

5. Core data contracts

```
// TaskEnvelope
{"episode_id":"uuid","brief":"...","task_family":"ai_badge_trace",
"apps":[{"name":"VectorCraft","version":"0.21.1","catalogue_sha256":"..."}],
"rights":{"source":"...","licence":"..."},
"output":{"format":"svg","width":2400,"height":3000,"dpi":300},
"gates":["craft_trace"],"budget":{"max_steps":20,"max_seconds":900,"max_cents":100}}
```

```
// ToolCall (idempotent)
{"tool":"vectorcraft.run","cmd_id":"imageTrace.make","params":{},
"idempotency_key":"sha256(episode|step|params|input_hashes)","dry_run":false}

// VerificationVerdict
{"rungs":{"process":{"ok":true},"artifact":{"sha256":"..."},"gates":{"ok":true},
"render":{"ok":true},"tiles":{"ok":true},"critic":{"verdict":"pass"},
"human":{"score":4}},"verdict":"accepted|rejected|needs_human|unverified"}

// Skill (procedural memory)
{"skill_id":"vc-ai-badge-trace","version":3,"app":"VectorCraft",
"catalogue_hash":"...","preconditions":{},"steps":[],"gates":[],
"stats":{"success":8,"failure":1},"status":"candidate|active|rolled_back"}
```

6. State machines

Episode

`planned -> executing -> verifying -> accepted | rejected | needs_human | unverified | failed` with crash recovery via the queue lease and idempotent tool ledger. A failed gate is terminal for shipping; retry only named transient faults.

Skill

`draft -> candidate -> active -> superseded | rolled_back`

Promotion requires: declared evaluation horizon, Wilson lower bound above baseline, zero golden regression, human approval. **3/3 is not sufficient** (Wilson LB 0.4385; the original rule was rejected by simulation). Demotion must be tested: "2 failures in last 20" false-demoted a 95% skill 41% of the time; prefer/test 2-consecutive or 3-in-20.

Job

`queued -> leased -> running -> done | failed | queued (retry) | dead`

Lease has `lease_until`; a reaper requeues expired work; bounded scans; no duplicate side effect.

7. Evidence ladder (why each rung exists)

Rung	Proves	Failure mode it catches
process	command ran, result shape	missing result, wrong exit
artifact/hash/dimensions	exact bytes/size/DPI	file exists but wrong size
domain gates	white residual, specks, coverage, text, fills	traces/gates failing
independent render	engine vs rsvg/PDF	renderer drift
tile/crop metrics	localized defects	global SSIM misses 4% defect
blind critic	brief/hierarchy/type/colour	ungrounded "looks good"
human rubric	taste/print safety	proxy optimization

Measured: global SSIM caught 7/15; tile-min 13/15 but 3/4 false alarms; the 5-check composite

15/15 recall, 0/4 false alarms. White fringe is invisible on white; render on black **and** white.

8. Hosting

Component	Bind	Notes
console	`127.0.0.1:8091`	existing; viewer only
DeerFlow	existing containers	DeepSeek key via env name; no secret in repo
Forge conductor	`127.0.0.1:8092`	systemd, loopback only
Forge UI	`127.0.0.1:8093`	optional approval viewer
`pod-mcp`	should be `127.0.0.1:8765` + auth	current `0.0.0.0` is a finding
Postgres	local container	new `forge` DB/role; `pod` untouched
artifacts	local content-addressed store	hash + gate report + retention

Capacity: 2 vCPU/7.9 GB, 36 GB free, no GPU; <=2 episodes, one global Craft semaphore, daily cost cap, nightly dump + restore drill, no new public port.

9. Failure and refusal behavior

Condition	Response
app binary missing/stub	`E_TOOL_NOT_INSTALLED`; stop
unknown command id	`E_COMMAND_UNKNOWN`; no fallback invocation
path outside roots	`E_ROOT_DENIED`
missing document/selection	open/select; `enabled:false` is state
PhotoCraft interleaved calls	await each reply; do not pipe dependent calls
gate failed / vacuous	`failed`/`unverified`; no ship
`prep_print_art` warning-only upscale	parse `warnings`; refuse print above limit
source mutated	fail/quarantine
key missing/invalid	named refusal; no fake completion
cap exceeded	stop before spend; `E_BUDGET`
worker crash	lease expiry/requeue; idempotent replay
critic/gate disagree	`needs_human`; critic cannot override
publish without approval	`E_PUBLISH_NOT_APPROVED`
app/catalogue drift	pin; skill expired until revalidated

10. Rollback

- Phase 0: restore original launchers, `routes.py`, MCP config from backup.
- Phase 1: stop/disable `forge-*`; `DROP DATABASE forge`; `pod` untouched; artifact store retained.
- Phase 2: `FORGE_MEMORY=off`; active skill pointer flips to previous version.
- Every file change records pre-change SHA-256; rollback is a file/pointer restore.

11. Build sequence

Step	Work	Evidence
1	I1 launchers/argv	one trace + one retouch
2	I2 MCP registration	`tools/list` + batch
3	I3 DDL/service/queue	apply twice; 33 tests; replay

```
| 4 | I4 verifier/critic | composite fixture results |
| 5 | I5 approval/publish | approval reuse blocked; read-back |
| 6 | I6 retrieval/promotion | A/B + null result handling |
| 7 | 50 goldens/print/second app | measured quality/cost |
```

12. What this architecture refuses to hide

- It cannot drive Film/Design/Effect today; those are absent.
- It cannot train weights or do RL on this VPS.
- It cannot prove "better than human" without a blind study.
- It cannot publish safely until templates/approval are configured.
- It cannot claim a live key until the zero-spend metadata check is run.

Related: [`README.md`](README.md) - [`execution-wave/09-GOAL-GAP-PLAN.md`](execution-wave/09-GOAL-GAP-PLAN.md) - [`execution-wave/10-CLAIM-REGISTER.md`](execution-wave/10-CLAIM-REGISTER.md) - [`diagrams/`](diagrams/index.html)

Goal evaluation and research plan

Date: 2026-11-05 - **Status:** evaluation is current; implementation is not started.

Companions: [`GOALS.md`](GOALS.md) -
[`execution-wave/09-GOAL-GAP-PLAN.md`](execution-wave/09-GOAL-GAP-PLAN.md) -

[`execution-wave/10-CLAIM-REGISTER.md`](execution-wave/10-CLAIM-REGISTER.md).

1. What we set out to achieve

Build an AI agent that can design and ship real Craft work, learn from its runs, remember what

works, use the existing VPS + DeepSeek V4.1 Flash, and be truthful about its limits.

2. Goal-by-goal outcome

```
| Goal | Desired outcome | Current outcome | Status | Evidence |

|---|---|---|---|---|

| **G1** Execute Craft | run trace/retouch/print through the pipeline | execution path is stubs/argv; real
CLIs work standalone | **BLOCKED** | `01-EXECUTION-REPAIR.md`; S1/S3 |

| **G2** Native tools | typed CLI/MCP surface, no GUI guessing | 918/965 commands; 33/26 MCP tools; not
registered; no gateway | **PARTIAL** | S3; `02-FORGE-SKELETON` |

| **G3** Verify | artifact + gates + render + blind critic + human | gates exist but can be
vacuous/pre-artifact; no integrated critic | **PARTIAL** | S3; `03-VISUAL-DEFECTS`; `04` |

| **G4** Learn | improve from verified episodes, remember | no episode/eval/skill tables; DeerMem facts
only; L0 | **NOT STARTED (L0)** | S1; S6; `04-PROMOTION-SIM` |

| **G5** Host | run on the existing VPS/key | DeerFlow 2.2 + `deepseek-flash` running; 2 vCPU/no GPU; key
unverified | **FEASIBLE** | S1; S12 |

| **G6** Publish safely | human-approved, verified product | no approval queue; templates unset; no
autonomous publish | **BLOCKED** | S1; S12 |

| **G7** Truth | labels, claims, gaps, no invention | 400 answers, claim register, errata; README claim
still needs correction | **IMPROVING** | REPORT; 10-claim; ERRATA |
```

3. What we actually know

Verified

- The VPS is 2 vCPU / 7.9 GB / 36 GB free, no GPU/display.

- DeerFlow 2.2.0 runs `deepseek-flash` ("DeepSeek V4.1 Flash"), 1,048,576 ctx, 8,192 output.

- VectorCraft 0.21.1 and PhotoCraft 0.20.2 are installed; their MCP servers work standalone.

- `/opt/pod-pipeline/bin/*-cli` are stubs exiting 127; `routes.py` argv is invented.

- Only `forge` MCP (3 pipeline tools) is registered in DeerFlow.

- `pod` DB is empty; no episode/eval/skill tables.

- 39 skills, 400 answers, 8 diagrams, 6 examples, 131-file package.

- Adapter: 30/30 tests. Promotion sim: 23/23 tests. Visual composite: 15/15 with 0/4 false alarms

on the synthetic corpus.

Not verified / not true

- The DeepSeek key's live validity.

- The README's live E2E claim (historical/unreproducible now).

- FilmCraft/DesignCraft/EffectCraft availability.

- Real gate/human rates, human inter-rater agreement, cost per accepted artifact.

- "The agent learns art direction" and "better than a human designer."

- A production DDL migration or live deployment.

4. What is possible now

1. **Fix the execution path** on a staging copy; run one real trace and one retouch; capture artifacts.

2. **Register the Craft MCP servers** and prove `tools/list`/`command_batch`.

3. **Run the Forge skeleton** DDL/tests against a scratch Postgres; prove duplicate/refusal/crash.

4. **Run the visual critic** on the 19-case synthetic corpus; require 15/15 and 0/4.

5. **Run the promotion simulation** and choose a policy; 3/3 is not a policy.

6. **Validate the key without spend** via `/user/balance` and `/models`.

5. What is not possible on this VPS now

- Driving FilmCraft/DesignCraft/EffectCraft: not installed.

- Fine-tuning/RL: API model, no GPU/open weights/corpus.

- Pixel-first GUI automation: no display; control bridge missing a library.

- Autonomous publish: no approval queue; template ids unset.

- A claim of artistic superiority: no blind study.

6. The best path (decision)

DeerFlow 2.2 as front door/runtime + Forge Conductor as a third plane + native Craft CLI/MCP + Postgres ledger/queue + DeerMem facts + versioned skills. Do not build a new generic harness; do not start with GUI or training; do not make the pipeline the agent.

7. Research continuation plan

These are new questions, not repeats of the 400.

#	Question	Method	Output	Stop condition
R1	Does the repaired launcher/argv execute a real trace?	staging copy + real CLI; artifact + JSON	`R1-RESULT.md`	first missing artifact
R2	Can DeerFlow load a stdio MCP server with roots?	register + `tools/list` + batch	`R2-RESULT.md`	handshake failure
R3	Does DDL apply twice and enforce transitions?	scratch PG16 + duplicate/refusal tests	`R3-RESULT.md`	SQL assertion fail
R4	Does the visual composite generalize?	new photographic/gradient corpus + human labels	`R4-RESULT.md`	false alarm > budget
R5	What promotion policy is defensible?	simulate n/k/p0, then real episodes	`R5-RESULT.md`	no separation from null
R6	Does retrieval improve first-attempt pass?	A/B 10-50 goldens, two arms	`R6-RESULT.md`	no gain -> disable
R7	Does the blind critic agree with humans?	blind ratings, precision/recall	`R7-RESULT.md`	low agreement -> needs_human
R8	What is the real cost per accepted artifact?	usage ledger + balance delta	`R8-RESULT.md`	budget cap exceeded
R9	Is the key valid without spend?	`/user/balance` + `/models`	`R9-RESULT.md`	401/402

| R10 | Can publish be made safe? | approval bind + read-back | `R10-RESULT.md` | any unapproved write |

8. Re-evaluation rule

Re-run this evaluation after every phase. A goal moves from `BLOCKED/PARTIAL` to `ACHIEVED` only when the named gate has a reproduced command/output and the claim register is updated. A null result is a valid outcome; a design is not an achievement.

9. Current verdict

Research/design: strong. Execution: blocked at Phase 0. Learning: not started. Product: feasible narrowly (VectorCraft + PhotoCraft) once the execution path and approval are real.

Plain English - what we have, what is wrong, and what we are building

This is the explanation for a human who does not want to read four hundred questions and a dozen evidence files. It is intentionally blunt. If something is not built, it says so.

1. What we are trying to do

We want an AI worker that can make real print/vector artwork using the Craft apps that already run on the server, check its own work with evidence, remember what worked, and get better under human supervision. It should be able to take a brief like "make this badge trace-ready and print-safe," choose the right Craft commands, produce an SVG/PNG/PDF, verify the size/DPI/colors/gates, show the result to a person, and only then publish.

That is the product. It is not "an AI that replaces the artist." It is a reliable production assistant with a paper trail.

2. What already exists

The server already runs **DeerFlow 2.2**, an agent runtime, with the **DeepSeek V4.1 Flash** model. DeerFlow can hold conversations, run subagents, remember facts across sessions, and connect to tools. That part is real.

The server also has **VectorCraft 0.21.1** and **PhotoCraft 0.20.2** installed. They are real Rust apps with 918 and 965 commands and 33/26 MCP tools. They are independent apps, not Adobe clones.

There is a console at port 8091 and a small pipeline that is supposed to run the apps, verify the image, and publish to Fourthwall. There are 39 skills, 400 research answers, 8 diagrams, and six worked examples.

That is a lot of real material. But having the parts is not the same as having a working machine.

3. What is actually broken

The pipeline is connected to **fake launchers**: the scripts under `/opt/pod-pipeline/bin` are tiny stubs that exit with code 127. The real Craft binaries are elsewhere and are never reached.

The Python `routes.py` also calls the apps with commands that do not exist. It pretends there is a `trace` subcommand and an `--input` argument. The real apps use `run --in ... --cmd ... --export ...`. So even if the stubs were replaced, the arguments would still fail.

The Craft MCP servers work when you start them by hand, but DeerFlow is not registered to use them. The only MCP server DeerFlow knows about is the old three-tool pipeline server.

The database is empty. There is no table that records an episode, a tool call, a gate result, a critique, an evaluation, or a skill. So the system cannot learn from its runs, because it does not

remember them.

There is no approval queue. The storefront product ids still contain `REPLACE_WITH_TEMPLATE_ID`. Publishing is therefore not safe and not configured.

Finally, the README says a live end-to-end run succeeded. That is not true today: the database is empty and the last real run failed at the first stub. Treat that sentence as history, not current status.

4. What we did about it

We did not start by rewriting the product. We did five things:

1. **Measured the real commands.** We ran the actual VectorCraft and PhotoCraft binaries and recorded the exact argv that works, including the fact that VectorCraft does not create export folders and PhotoCraft does.
2. **Designed the conductor.** We designed a third control plane, "Forge Conductor," that sits beside the console and the pipeline. It owns planning, tool calls, evidence and learning; it does not own the Craft apps' document truth or the model weights.
3. **Built executable evidence.** A 13-table Postgres schema, a 36-edge state machine, 33 Python tests, an adapter with 30 tests including a real crash/kill test, and a visual corpus with 15 defects and 4 controls.
4. **Corrected our own mistakes.** The original promotion rule ("3 successes + Wilson 0.60") cannot work: 3/3 gives a lower bound of 0.4385. The simulation replaced it with a horizon-based policy, and we wrote the correction into the skills instead of hiding it.
5. **Packaged everything.** The corpus, skills, examples and diagrams are in a 131-file Desktop folder and mirrored into the vault, with hashes and an index. Nothing was deleted.

5. The six integrations, in plain terms

I1 - Fix the launchers and commands. Point the pipeline at the real apps and use the commands that actually exist. This is the first thing to build. Without it, nothing else can run.

I2 - Connect the real Craft MCP servers. Register VectorCraft and PhotoCraft with DeerFlow, give them explicit read/write folders, and wait for each call to finish. PhotoCraft can interleave calls; a dependent call must not be fired blindly.

I3 - Add the conductor. A small service that receives a task, makes a plan, runs the steps, records everything, and can recover after a crash. It uses the new Postgres tables and a queue.

I4 - Add the verifier and blind critic. A step is not done because the command exited 0. The verifier checks the file hash, dimensions, DPI, gates, a separate render, tile/crop differences, and a critic that sees the image without knowing which version is supposed to be better. Missing evidence means "unverified," not "pass."

I5 - Add human approval before publish. Approval is tied to the exact artifact hash. A stale approval cannot be reused. The storefront product/template is checked before any write. No autonomous publishing.

I6 - Add learning and evaluation. Episodes become searchable cases; repeated success can become a versioned skill; a skill is promoted only after a declared number of trials, a Wilson lower bound above baseline, a golden regression check, and human approval. A null A/B result disables the memory. No fine-tuning or RL on this server.

6. The eight Python changes, in plain terms

1. Rewrite `routes.py` to call the measured commands.
2. Add a preflight that checks "is the app installed, is the version/catalogue pinned?" and refuses by name if not.
3. Fix `craft_trace` so the text check cannot pass on zero boxes and the artifact is not saved before the gate passes.
4. Fix `prep_print_art` so `ok:true` is not treated as a pass and a 3.6x upscale is not silently called print-safe.
5. Hash the source before and after; fail if the run mutated the original.
6. Add idempotency and duplicate replay to the tool gateway/DB.
7. Add a worker lease and reaper so a crash requeues work exactly once.
8. Record token/cost per step and enforce a cap.

These are small, testable changes. They are not a rewrite.

7. The eight agent/skill changes, in plain terms

1. The Craft operator must attach evidence, not say "looks good."
2. The learning skill must use calibrated promotion and tested demotion, not 3/3.
3. The visual critic must use the five-check composite and render on black and white.
4. The harness architect must check which apps exist before designing for them.
5. The code adapter must record donor provenance and not copy code verbatim.
6. The research navigator must point to the right slice of the 400 answers without flooding context.
7. The MCP skill must wait for dependent calls and be honest about stdio vs HTTP.
8. The print/publish skill must bind approval to the artifact hash and check rights/templates.

8. What the goals are - and where they stand

- **Execute Craft work:** blocked. The stubs and wrong arguments stop it.
- **Use native tools:** partial. The MCP servers exist; they are not registered.
- **Verify quality:** partial. Gates exist but can lie; the critic is not wired in.
- **Learn:** not started. There is no episode/eval/skill data.
- **Host on the VPS:** feasible. The runtime and model are there; it is constrained.

- **Publish safely:** blocked. No approval, no template ids.
- **Be truthful:** improving. We have a claim register and an errata file; the old README sentence still needs correcting in the product docs.

9. What is possible, and what is best

Possible now: fix the execution path, run one real trace and one real retouch, record the evidence, build the ledger and queue, run the visual critic on the synthetic corpus, and validate the key without spending money.

Not possible now: all five Craft apps (three are absent), autonomous publishing, training new model weights, RL, GUI-first automation (no display), or proving "better than a human."

Best path: fix I1 and I2 first, then I3 and I4. Do not start with a GUI agent, a training run, or a redesign. The bottleneck is the execution path, not the model.

10. What we need from Riley

1. Approve Phase 0 and let the repair be applied to a **staging copy** first.
2. Approve the zero-spend key check (`/user/balance`, `/models`); do not use a chat completion as a key test.
3. Decide whether FilmCraft/DesignCraft/EffectCraft will be purchased; until then they are out of scope.
4. Set the Fourthwall template/area ids or confirm publish stays disabled.
5. Choose the promotion baseline (`p0`), the evaluation horizon (`n`) and the false-promotion budget; the simulation is a suggestion, not a policy.
6. Name the human approver for taste/print/publish.

11. The honest bottom line

We have a strong research base, a real runtime, a real command catalogue, a real execution plan, and working prototype tests. We do **not** have a working end-to-end Craft agent yet. The first honest milestone is one real trace and one real retouch, recorded with evidence. After that, the learning and publishing layers are worth building.

12. Where to read more

- [`README.md`](README.md) - the map and the plans.
- [`ARCHITECTURE.md`](ARCHITECTURE.md) - the system design.
- [`GOALS.md`](GOALS.md) - goals and non-goals.
- [`GOAL-EVALUATION.md`](GOAL-EVALUATION.md) - goal-by-goal status.
- [`execution-wave/09-GOAL-GAP-PLAN.md`](execution-wave/09-GOAL-GAP-PLAN.md) - the decision-complete plan.
- [`execution-wave/10-CLAIM-REGISTER.md`](execution-wave/10-CLAIM-REGISTER.md) - what is proven.

- [`diagrams/index.html`](diagrams/index.html) - eight pictures.
- [`PDF-CATALOGUE.md`](PDF-CATALOGUE.md) - the PDF reading copies.